

Understanding the Leaper experiment plan

1. What we are trying to find out

Can Leaper search more effectively if it learns to remember its own experience? That is the main question. We also need to check whether its problems come from the information we give it, the rewards, or getting stuck while moving.

The task has three parts: look for the hidden target, move toward it once discovered, and avoid getting trapped along the way. A model can improve at one part while getting worse at another. That is why a single success score does not explain everything.

What Leaper has now

The current model is an **MLP**, short for *multilayer perceptron*. It takes a list of numbers describing the current situation and uses them to choose movement. Its own internal state does not persist from one decision to the next.

But it does receive information stored by the simulation. This is **maintained memory**: code keeps records and supplies useful summaries to the model. The important records are:

- **Target memory**: where the target was last seen and how long ago. The model can use that location even after the target leaves view.
- **Search memory**: which areas have been visually checked. The model receives a compact suggestion about an unchecked region, rather than the whole map.
- **Recent context**: its previous action and whether it collided.

So "the MLP has no internal memory" does **not** mean "Leaper gets no information about the past."

What an LSTM would add

An **LSTM**, short for *long short-term memory*, is a network component that carries a learned summary from one decision to the next. It could learn to retain useful clues about routes, unsuccessful actions, and earlier observations. We have to test whether it actually does.

The proposal is to keep the existing movement model and add a small LSTM component. At the beginning, that new component contributes nothing to the action, so the model starts with the old behavior. Training can then teach it useful changes. The old part can also change during training, so this does not guarantee its skills will remain perfect.

The first memory experiment keeps the existing notes. Only after learned memory helps do we test whether it can replace the search suggestion.

Four terms worth knowing

Parameter: an adjustable number in the network. Training changes these numbers. More parameters allow more complicated behavior, but do not automatically produce better behavior.

Node or unit: one small calculation inside the network. A "64-unit LSTM" is not a map with 64 locations or a guarantee that it remembers 64 events. **Layer** means one stage of network processing.

Note: our informal name for input information. It is not a node and is not a written instruction. A frontier note contains direction, distance, the size of an unchecked region, and whether the suggestion is valid.

State augmentation: adding useful information to the model's inputs. For example, adding a summary of previously checked areas is state augmentation.

2. Why the reward has not solved everything

The model does not reason in words: "The target is worth 25 points, so obviously I should go there." Training adjusts its parameters based on the experiences it collects. It must experience enough useful actions and learn which ones lead to later success.

What the rewards already do

Before the target is discovered, Leaper can earn small bonuses for new areas or views. First discovery earns **+0.5**. After discovery, getting closer earns a progress reward, and moving farther away can lose points. Arrival earns **+25**. Time, collisions, and prolonged inactivity have costs.

The exploration bonuses already stop after discovery. The search suggestion can still be present, but the model is no longer being paid those bonuses for exploring. Changing what it sees and changing what earns points are two different interventions.

The code also gives some pre-discovery reward for getting closer to the hidden target. The model is not directly given the target's coordinates, but this reward uses information the simulator knows. One planned experiment removes that reward to see whether it is helping useful search.

Why a large reward can still be hard to learn from

Suppose one route eventually reaches the target but needs a long detour around rocks. Early actions on that route may look unhelpful until the model has learned what follows. If it rarely finishes the route, the training evidence for those actions is weak.

There is also **discounting**: the training method gives more weight to rewards received sooner. With the current setting, +25 received 200 decisions later contributes roughly 9.2 to the present decision's discounted score. At 500 decisions later, it contributes about 2.0. The arrival reward is still +25; these smaller numbers describe how training values that distant reward today.

This is why "make the final reward large" is useful but insufficient. We need successful experience, helpful information, and movement that can complete the route.

What the existing tests tell us

An existing test removed the search suggestion after discovery. Across six saved models, overall arrival success rose from about **71.2% to 72.7%**: a gain of **1.5 percentage points**, or roughly one to two extra successes per hundred attempts. One of the six models got worse. This is a small effect, not an explanation for all the failures.

The same test also recorded why attempts ended. Among 600 normal-condition attempts, there were 427 arrivals and 173 failures. Of those failures, 149 ended because Leaper got stuck or stopped moving long enough to trigger an early failure. Only 24 reached the time limit. These attempts reuse maze layouts across models, so they are not 600 independent mazes.

We therefore need to measure **finding the target**, **reaching it after discovery**, and **getting stuck** separately. The old headline success score cannot tell us which of these to fix.

Why revisit LSTM after its earlier failures?

The earlier LSTMs were much larger and started from scratch. One later attempt also changed rewards and training settings. Those results show that those setups did not work well. They do not cleanly answer whether a small memory component added to the existing model would help.

The earlier PDF's preference for maintained memory was sensible: explicit records are easier to inspect, and preserving working movement reduces the learning burden. The new plan keeps that principle while giving learned memory a fair comparison.

3. The first four training comparisons

We should not start by changing everything. Begin with four models that receive the same information and face the same task. An **experiment** changes a specific part of the setup to test an explanation. A **control** is the comparison model that helps show whether that change mattered.

Experiment	What changes?	What it tells us
03: Continue the MLP	Give the existing model more training	Could extra practice solve the problem without internal memory?
04: Larger MLP	Add more calculations, but no internal memory	Would a bigger model help even without remembering a sequence?
05: Short history	Show a few earlier observations alongside the current one	Is recent history enough?
06: Small LSTM	Add a learned summary that carries forward	Does learned memory help beyond those simpler alternatives?

The experiment numbers match the detailed report. Experiments 01 and 02 are checks that come before this training group.

How large are these models?

The continued MLP has about **13,000 parameters**. The larger MLP and the small LSTM each have about **73,000**. Keeping those two close in size helps separate "memory helped" from "a larger network helped." The short-history model has about **48,000**.

For comparison, the earlier search LSTM had about 623,000 parameters. The proposed first LSTM is substantially smaller. We start with one recurrent layer containing 64 units; we do not immediately add multiple layers.

Each model will also receive its actual recent movement: how far it moved and how much it turned. This is called **odometry**. It matters because trying to move into a rock and actually moving are different experiences. Giving these measurements to all four models keeps the comparison fair.

What is PPO, and what are the actor and critic?

PPO, short for *proximal policy optimization*, is our training method. It learns from actions that worked better than expected, while limiting how sharply the model changes in each update.

The **actor** chooses movement. The **critic** estimates future reward to help training judge actions. It is another learned calculation, not another character in the game.

The actor and critic get separate memory components. We are not adding separate search and chase controllers with a rule that switches between them.

What does a training budget mean?

A **step** is one decision and its result in one simulated environment. **100k steps** means about 100,000 such experiences, not 100,000 complete attempts. An **episode** is one attempt, from its starting position until success, failure, or the time limit.

Start with a short check that the code works, then inspect learning at about 100k steps. Give the main LSTM at least **500k additional steps** before its first serious judgment. Train its main controls for the same duration. The short-history model gets 250k initially and more if it remains promising.

These steps build on earlier training. No fixed count guarantees enough practice: extend improving models and their controls fairly.

4. The other experiments, in ordinary language

The full plan has **15 entries: two checks and 13 training comparisons**. The four central training comparisons were explained on the previous page. The remaining entries answer more specific questions. We run them in stages rather than launching all fifteen together.

Checks before training

01. Verify the setup. Check that models load, inputs mean what we think they mean, and memory resets between attempts. Measure training speed on the processor and graphics card. A coding error in memory handling can make a useful design appear useless.

02. Explain the failures. Reuse the existing search-note test and check its details. Separate failures before and after discovery, and distinguish collisions, prolonged stopping, and time limits. This is evaluation: testing saved models, without teaching them anything new.

Follow-ups to the memory comparison

07. Try a GRU. A GRU, or *gated recurrent unit*, is another learned-memory design with a simpler internal structure than LSTM. Test whether it works as well at lower cost.

08. Try a larger LSTM. Increase from 64 to 128 units, while keeping one layer. Give it about one million additional steps and compare the smaller LSTM at the same training duration. Run this only if more capacity looks worth investigating.

09. Train on longer sequences. A sequence is an ordered stretch of experience. Compare learning from stretches of up to 128 versus 256 steps. This changes how far back training can directly connect actions and observations within that stretch. It does not force the model to forget after 128 steps during play.

10. Remove the search suggestion. After a useful LSTM has been found, train one copy with the suggestion removed and another with it retained. Keep the remembered target location. This tests replacing one maintained-memory aid at a time.

11. Ask memory to predict searched areas. Add a training exercise: predict which nearby areas have already been observed. This is an **auxiliary task**, meaning an extra learning exercise intended to improve information inside the model. It does not tell the model which action to take, and it is not an extra gameplay reward.

Follow-ups to information and rewards

12. Reward newly seen space. Try a small, capped bonus for revealing previously unseen space, replacing the existing new-cell and new-view bonuses. Walking somewhere new is not always the same as getting a useful new view. Keep this reward off after discovery.

13. Suggest a better viewing position. A **frontier** is the boundary between checked and unchecked space. Instead of pointing toward an unchecked location that may be inside a rock, suggest an observed-free position from which Leaper could inspect that area. The suggestion is still numbers describing a possible destination; Leaper still learns whether to follow it. Do not give it an all-knowing route through unseen obstacles.

14. Compare Tanh and ReLU. These are **activation functions**: rules used inside a network that help it learn more than a simple straight-line relationship. Tanh smoothly limits a calculation to between -1 and +1. ReLU sets negative values to zero and leaves positive values unchanged. Neither is automatically better. Change this only in the added MLP component, keeping the comparison controlled.

15. Remove the hidden-distance bonus. Before discovery, stop giving points merely because Leaper moved closer to the target's true location. Keep the rewards after discovery. This tests whether that unseen-target training hint helps search or distracts from it.

5. How long, and what counts as success?

What the machine can do

The graphics card is an **RTX 3050 Laptop with 4 GB of memory**. **PyTorch**, our neural-network software, currently uses only the CPU, or main processor. GPU training needs a compatible installation that can use the graphics card.

A GPU can accelerate network calculations, but movement and visibility checks also take time. Measure the full training process before promising faster runs.

Training amount	Historical MLP speed	Historical larger-LSTM speed
About 100k steps	About 6 minutes	About 20 minutes
About 250k steps	About 14 minutes	About 47 minutes
About 500k steps	About 28 minutes	About 94 minutes
About 1 million steps	About 55 minutes	About 187 minutes

These estimates use old run speeds. New models, setup work, and extra evaluation can change the total time. Measure the new setups before scheduling them.

How we avoid being fooled by one good run

A **seed** reproduces an experiment's random choices. One lucky run can give a misleading result. Train the promising designs three times with different training seeds.

Compare models on the same mazes. Then test the finalist on 1,000 reserved mazes that were not used to choose it.

Deterministic evaluation uses the model's usual selected action. **Stochastic evaluation** samples actions with variation. Report them separately: random variation during training can help a model escape situations where its usual action fails.

The immediate target

Keep the stage requirement: average arrival success of at least **78%**, and first detection in at least **90%** of attempts, across three training runs. Confirm on the reserved mazes, and beat the MLP trained for the same duration. Passing is not a promise of perfect behavior.

For example, 90 targets found but only 78 reached means 12 failures after discovery and ten without discovery. Investigate those separately, and check for increases in stuck or frozen behavior.

What we do with the result

If LSTM wins, test which maintained notes it can replace. If the larger MLP or short history matches it, use that simpler alternative. If viewing suggestions or rewards win, improve those before enlarging memory.

Introduce slow target movement after stationary seeking qualifies. Previously checked areas could then contain the target again, so remembering when something was seen becomes important. A competing, learning hider comes later.

Start by checking the setup, then run the four-model comparison. Test whether memory earns its added complexity. No new training has been launched for these reports.

This is the plain-language companion to Leaper Seeker Experiment Plan. The detailed report contains exact settings, implementation checks, experiment budgets, and research references. Experiment numbers are unchanged.